

AI Lab Project Guide (UI + AI Integration)

This document explains exactly what students need to build in their selected lab project.

1. Project Goal

Build a Python application with a **visual user interface** (UI) that solves an AI-related problem from your lab work.

The app can be:

- A **web app** (recommended: Streamlit, Flask, FastAPI + frontend)
- A **desktop app** (Tkinter, PyQt, Kivy, etc.)

Your UI must not be text-only. It must visually represent data, process, and results.

2. What You Must Deliver

Each student/group must submit:

1. Complete source code
2. `README.md` with setup/run instructions
3. Requirements file (`requirements.txt`)
4. Sample dataset(s) used by your app
5. Screenshots (or short video) of working UI
6. Short report (problem, method, AI used, results)

3. Minimum Required Features

Your app must include all of the following:

A) Problem Setup Module

- Define the problem clearly (input, output, constraints).
- Let user provide or select input data.
- Validate user input and show clear error messages.

B) Core Logic Module

- Implement the main algorithm/rules/model for your project.
- Keep code modular (separate logic from UI).

- Show intermediate steps where possible (not only final output).

C) Visual UI Module (Compulsory)

Your UI should display data visually using one or more of these:

- Charts (bar/line/pie/scatter)
- Tables with highlights
- Graph/network view
- Grid/map/path view
- Timeline/step-by-step animation

Also include:

- Controls (buttons, dropdowns, sliders, toggles)
- Result panel (final answer + explanation)
- Status messages (success/error/loading)

D) Explainability Module

- Show why the app produced that output.
- Display key factors/features/rules used in decision.
- Add a short natural-language explanation for users.

E) Evaluation Module

- Show at least 2-3 performance indicators (as applicable), for example:
 - Accuracy / precision / recall
 - Cost / path length / expanded nodes
 - Runtime / memory
- Compare at least two settings, baselines, or approaches.

4. Suggested Function Structure (Reference)

Use similar function-level organization in your code:

- `load_data(path)`
- `preprocess_data(data)`
- `run_model_or_algorithm(data, params)`
- `generate_explanation(result, context)`
- `create_visuals(data, result)`
- `render_ui()`

You may rename functions according to your project, but keep the same modular idea.

5. How to Add AI in the Project

You can integrate AI in multiple ways. Pick what matches your topic.

Option 1: Rule-based AI

- Facts + rules + inference
- Example: forward chaining to derive decisions

Option 2: Search/Optimization AI

- BFS, DFS, UCS, A*, greedy search, local search
- Visualize explored states and final path/solution

Option 3: Machine Learning AI

- Train a model on dataset (classification/regression/clustering)
- Show training results and predictions visually

Option 4: NLP/LLM-assisted AI (optional advanced)

- Add chatbot-style explanation panel
- Summarize input/output in plain language
- Provide recommendation from model output

Important: If you use external AI APIs, clearly mention:

- Which API/service was used
- What prompt/input was sent
- What output was returned
- Any limitations/cost/privacy concerns

6. UI Quality Checklist

Before submission, verify:

- UI is clean and readable
- Colors/fonts are consistent
- Works on expected screen size
- Buttons/controls are understandable
- Graphs have labels and titles
- User can run complete flow without code edits

- User can run complete flow without code edits

7. Recommended Tech Choices

Fastest path (recommended)

- Python + Streamlit + Plotly/Matplotlib

Desktop option

- Python + PyQt/Tkinter + Matplotlib

For model training

- scikit-learn / pandas / numpy

8. Example User Flow

1. User opens app
2. User uploads/selects data
3. User chooses algorithm/model settings
4. User clicks run
5. App shows:
 - Visual results
 - Metrics
 - Explanation
6. User compares alternative settings

9. Coding Standards

- Use meaningful variable/function names
- Keep functions short and reusable
- Avoid hardcoded paths
- Add comments only where logic is non-obvious
- Handle exceptions gracefully

10. Viva/Presentation Questions (Prepare These)

Be ready to explain:

1. Why this problem is relevant
2. Why this algorithm/model was selected

3. What data is used and how it is processed
4. How UI helps understand the result
5. What AI component is added and why
6. Limitations and future improvements

11. Suggested Marking Rubric (100)

- Problem understanding and design: 15
- Correctness of core implementation: 25
- AI integration quality: 20
- UI/visualization quality: 20
- Explainability and evaluation: 10
- Code quality + documentation: 10

12. Submission Naming Format

Use this structure:

- `ProjectName_GroupName/`
 - `app.py` or main entry file
 - `requirements.txt`
 - `README.md`
 - `data/`
 - `screenshots/`

13. Final Note for Students

Your project should not only produce a result; it should help a user **see and understand** how the AI system reached that result.

If your app is visual, interactive, and explainable, you are on the right track.